

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE-CSE=AI	Date:	18-07-2026

Code of Conduct

I S.Muskan(192472182) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Muskan

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Question 1: Intelligent Email, Password and Mobile Validator using Regular Expressions

Problem Understanding

- Accept Email, Password, and Mobile Number from the user.
- Validate each input using Python Regular Expressions.
- Ensure the email starts with a letter and contains a valid domain extension.
- Verify that the password meets all security conditions.
- Check whether the mobile number contains exactly 10 digits and starts with a digit between 6 and 9.
- Display the validation result for each input separately.

Algorithm

1. Import the **re** module.
2. Read Email, Password, and Mobile Number.
3. Define separate regular expression patterns for email, password, and mobile number.
4. Use **re.fullmatch()** to compare each input with its pattern.
5. Print whether each input is valid or invalid.

Python Code

```
import re

mail = input("Enter Email Address: ")
pwd = input("Enter Password: ")
phone = input("Enter Mobile Number: ")

mail_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
pwd_pattern = r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#%&!]).{8,}$"
phone_pattern = r"^[6-9][0-9]{9}$"

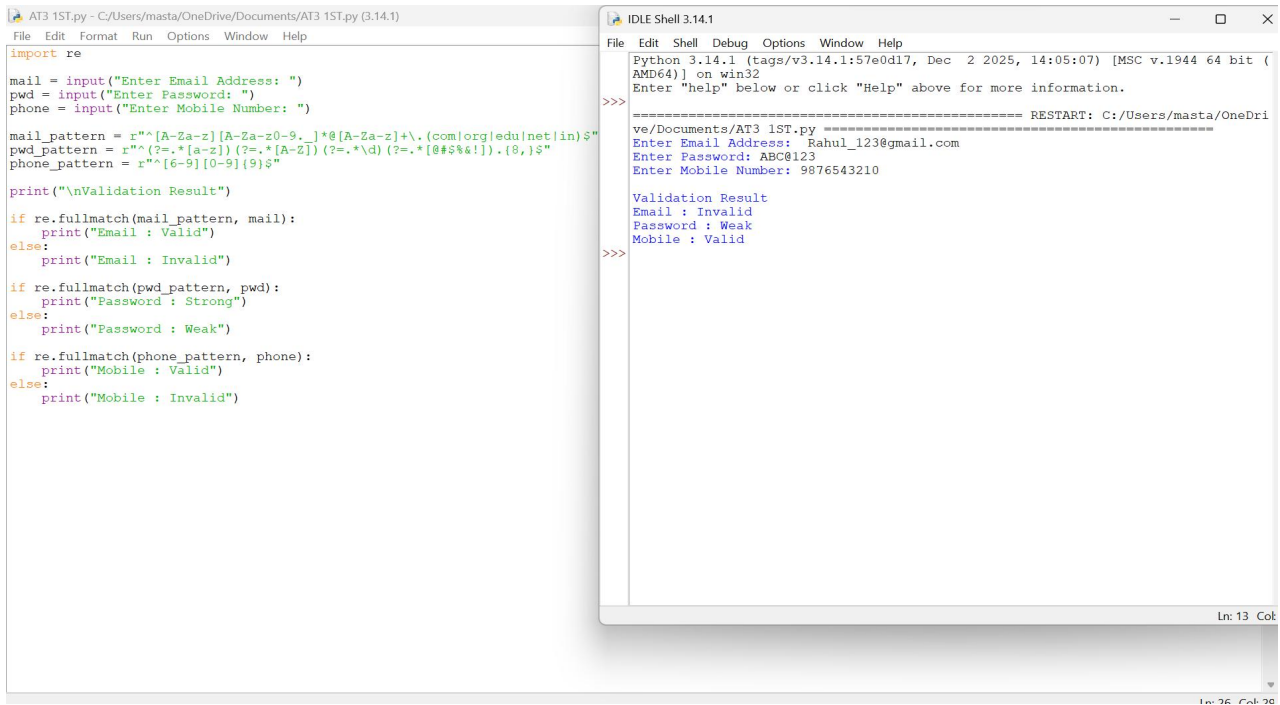
print("\nValidation Result")

if re.fullmatch(mail_pattern, mail):
    print("Email : Valid")
else:
    print("Email : Invalid")

if re.fullmatch(pwd_pattern, pwd):
    print("Password : Strong")
else:
    print("Password : Weak")

if re.fullmatch(phone_pattern, phone):
    print("Mobile : Valid")
else:
    print("Mobile : Invalid")
```

Sample Output



The image shows a screenshot of a Python script and its execution in IDLE Shell. The script, named 'AT3 1ST.py', is located at 'C:/Users/masta/OneDrive/Documents/AT3 1ST.py (3.14.1)'. It uses the 're' module for regular expressions to validate email addresses, passwords, and mobile numbers. The script prompts the user to enter an email address, password, and mobile number. The execution in IDLE Shell shows the user inputting 'Rahul_123@gmail.com' for the email address, 'ABC@123' for the password, and '9876543210' for the mobile number. The output shows the validation results: 'Email : Invalid', 'Password : Weak', and 'Mobile : Valid'.

```
import re

mail = input("Enter Email Address: ")
pwd = input("Enter Password: ")
phone = input("Enter Mobile Number: ")

mail_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
pwd_pattern = r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#%&!]).{8,}$"
phone_pattern = r"^[6-9][0-9]{9}$"

print("\nValidation Result")

if re.fullmatch(mail_pattern, mail):
    print("Email : Valid")
else:
    print("Email : Invalid")

if re.fullmatch(pwd_pattern, pwd):
    print("Password : Strong")
else:
    print("Password : Weak")

if re.fullmatch(phone_pattern, phone):
    print("Mobile : Valid")
else:
    print("Mobile : Invalid")
```

```
Python 3.14.1 (tags/v3.14.1:57e0d17, Dec 2 2025, 14:05:07) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/masta/OneDrive/Documents/AT3 1ST.py =====
Enter Email Address: Rahul_123@gmail.com
Enter Password: ABC@123
Enter Mobile Number: 9876543210

Validation Result
Email : Invalid
Password : Weak
Mobile : Valid
>>>
```

Conclusion

The application efficiently validates email addresses, passwords, and mobile numbers using Regular Expressions. It checks all required constraints and provides accurate validation results.

Question 2: Design a Finite State Automata (DFA) Simulator

Problem Understanding

- Create a Deterministic Finite Automata (DFA) simulator.
- Accept multiple input strings from the user.
- Simulate state transitions according to the transition table.
- Display the transition path.
- Indicate whether the input string is Accepted or Rejected based on the final state.

Algorithm

1. Define DFA states and transition table.
2. Set the initial state and final state.
3. Read the input string.
4. Process each symbol one by one.
5. Move to the next state using the transition table.
6. Store the visited states.
7. After processing all symbols, check whether the current state is a final state.
8. Display the transition path and acceptance result.

Python Code

```
transition = {
    ("q0", "a"): "q1",
    ("q0", "b"): "q0",
    ("q1", "a"): "q1",
    ("q1", "b"): "q2",
    ("q2", "a"): "q1",
    ("q2", "b"): "q0"
}
start = "q0" final = ["q2"]
string = input("Enter Input String: ")
state = start path = [state]
for ch in string:
    if (state, ch) in transition:
        state = transition[(state, ch)]
        path.append(state)
    else:
        print("Invalid Input")
        exit()
print("Transition Path:") print(" -> ".join(path))
if state in final:
    print("Accepted") else:
    print("Rejected")
```

Sample Output

The image shows a Python IDE with two windows. The left window displays a script for a DFA simulator. The script defines a transition table, initializes the start and final states, and processes an input string by following the transitions. The right window shows the execution output, including the input string 'abaabab', the transition path 'q0 -> q1 -> q2 -> q1 -> q1 -> q2 -> q1 -> q2', and the final result 'Accepted'.

```
AT3 2ND.py - C:/Users/masta/OneDrive/Documents/AT3 2ND.py (3.14.1)
File Edit Format Run Options Window Help

transition = {
    ("q0", "a"): "q1",
    ("q0", "b"): "q0",
    ("q1", "a"): "q1",
    ("q1", "b"): "q2",
    ("q2", "a"): "q1",
    ("q2", "b"): "q0"
}

start = "q0"
final = ["q2"]

string = input("Enter Input String: ")

state = start
path = [state]

for ch in string:
    if (state, ch) in transition:
        state = transition[(state, ch)]
        path.append(state)
    else:
        print("Invalid Input")
        exit()

print("Transition Path:")
print(" -> ".join(path))

if state in final:
    print("Accepted")
else:
    print("Rejected")

IDLE Shell 3.14.1
File Edit Shell Debug Options Window Help

Python 3.14.1 (tags/v3.14.1:57e0d17, Dec 2 2025, 14:05:07) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/masta/OneDrive/Documents/AT3 2ND.py =====
Enter Input String: abaabab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2 -> q1 -> q2
Accepted
>>>
```

Conclusion

The DFA simulator successfully processes input strings, displays every state transition, and determines whether the string belongs to the language accepted by the automaton.

Question 3: Smart Pattern Matching Engine using Regular Expressions

Problem Understanding

- Read a text paragraph from the user.
- Provide a menu to search different patterns.
- Detect dates, phone numbers, hashtags, mentions, prefixes, and suffixes using Regular Expressions.
- Display every matching result found in the text.

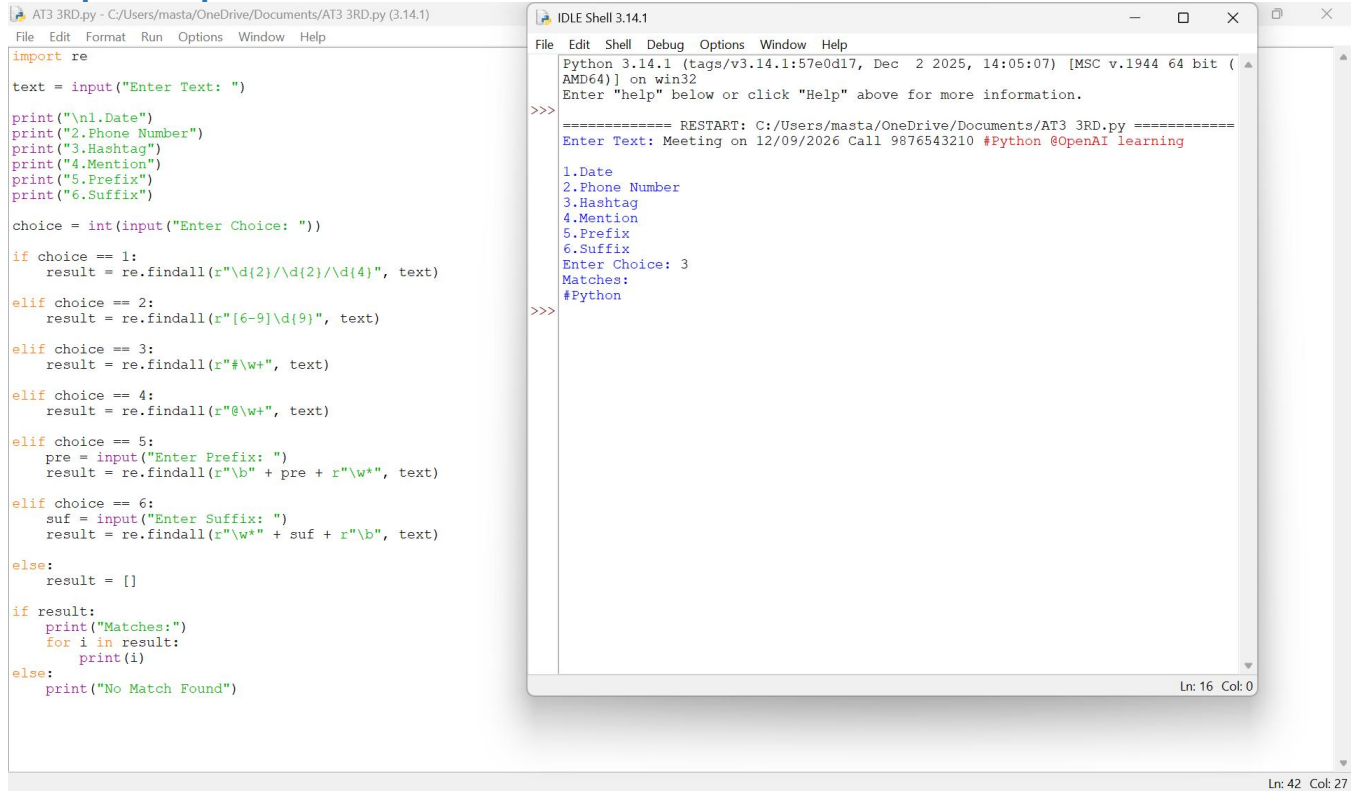
Algorithm

1. Import the **re** module.
2. Read the complete text.
3. Display the search menu.
4. Read the user's choice.
5. Select the corresponding Regular Expression pattern.
6. Use **re.findall()** to search for matches.
7. Display all matched patterns or print "No Match Found."

Python Code

```
import re
text = input("Enter Text: ")
print("\n1. Date")print("2. Phone
Number")print("3. Hashtag")print("4. Mention")print("5. Prefix")print("6. Suffix")
choice = int(input("Enter Choice: "))
if choice == 1:
    result = re.findall(r"\d{2}/\d{2}/\d{4}", text)
elif choice == 2:
    result = re.findall(r"[6-9]\d{9}", text)
elif choice == 3:
    result = re.findall(r"#\w+", text)
elif choice == 4:
    result = re.findall(r"@w+", text)
elif choice == 5:
    pre = input("Enter Prefix: ")
    result = re.findall(r"\b" + pre + r"\w*", text)
elif choice == 6:
    suf = input("Enter Suffix: ")
    result = re.findall(r"\w*" + suf + r"\b", text)
else:
    result = []
if result:
    print("Matches:")
    for i in result:
        print(i)
else:
    print("No Match Found")
```

Sample Output



The image shows a screenshot of a Python script and its execution output. The script, named `AT3 3RD.py`, is located at `C:/Users/masta/OneDrive/Documents/AT3 3RD.py (3.14.1)`. It uses the `re` module to parse a text input based on a menu choice. The output window shows the script being restarted and the results of parsing the input text: "Meeting on 12/09/2026 Call 9876543210 #Python @OpenAI learning".

```
AT3 3RD.py - C:/Users/masta/OneDrive/Documents/AT3 3RD.py (3.14.1)
File Edit Format Run Options Window Help

import re

text = input("Enter Text: ")

print("\n1.Date")
print("2.Phone Number")
print("3.Hashtag")
print("4.Mention")
print("5.Prefix")
print("6.Suffix")

choice = int(input("Enter Choice: "))

if choice == 1:
    result = re.findall(r"\d{2}/\d{2}/\d{4}", text)

elif choice == 2:
    result = re.findall(r"[6-9]\d{9}", text)

elif choice == 3:
    result = re.findall(r"#\w+", text)

elif choice == 4:
    result = re.findall(r"@w+", text)

elif choice == 5:
    pre = input("Enter Prefix: ")
    result = re.findall(r"\b" + pre + r"\w+", text)

elif choice == 6:
    suf = input("Enter Suffix: ")
    result = re.findall(r"\w*" + suf + r"\b", text)

else:
    result = []

if result:
    print("Matches:")
    for i in result:
        print(i)
else:
    print("No Match Found")

IDLE Shell 3.14.1
File Edit Shell Debug Options Window Help

Python 3.14.1 (tags/v3.14.1:57e0d17, Dec 2 2025, 14:05:07) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/masta/OneDrive/Documents/AT3 3RD.py =====
Enter Text: Meeting on 12/09/2026 Call 9876543210 #Python @OpenAI learning

1.Date
2.Phone Number
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 3
Matches:
#Python

>>>
```

Conclusion

The pattern matching engine efficiently searches different types of text patterns using Regular Expressions. It provides a simple menu-driven interface and accurately extracts the required information from the given text.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____